

1. 研究背景與策略概念

1.1 商品（金屬）在投資組合中的角色

商品資產（Commodities）在投資組合中扮演重要的多元分散與風險對沖角色，特別是在通膨上升或經濟循環變動時，商品價格通常與股票與債券呈現較低相關性。金屬類商品如黃金、白銀、銅、白金與鈀金，除了具有金融屬性外，也與工業需求密切相關，因此能反映總體經濟與產業循環狀況。將金屬納入投資組合，有助於提升資產配置的分散效果並降低整體投資風險。

1.2 動量策略與輪動策略的基本概念

動量策略（Momentum Strategy）是一種基於資產過去報酬趨勢進行投資決策的策略，假設過去表現較佳的資產在短至中期內仍可能持續表現良好。相關研究顯示，動量效應廣泛存在於股票、債券、外匯與商品市場。

輪動策略（Rotation Strategy）則是根據不同資產的相對表現動態調整投資組合配置，將資金配置至表現較強勢的資產，同時降低對弱勢資產的曝險。結合動量訊號的輪動策略可提升資產配置效率，並在市場循環中捕捉超額報酬。

1.3 本研究目標

本研究旨在建立一套基於動量與波動率配置之五大金屬輪動投資策略（Momentum and Volatility-Based Rotation Strategy for Five Major Metals），透過歷史價格資料進行回測，以評估策略之長期投資績效與風險特性。研究重點包括策略之報酬表現、最大回撤、夏普比率以及交易行為分析，並進一步探討策略在不同市場環境下的風險與改進方向。

2. 策略設計

2.1 資料與標的

本研究選取五種主要金屬商品作為投資標的，分別為黃金（Gold）、白銀（Silver）、銅（Copper）、白金（Platinum）與鈀金（Palladium）。此五種金屬涵蓋貴金屬與工業金屬，具有不同的金融與經濟循環特性，能夠代表商品市場的多元結構。

研究使用每日價格資料作為回測基礎，包含開盤價、收盤價以及相關技術指標。為了判斷長期趨勢，本研究計算 200 日指數移動平均線（Exponential Moving Average, EMA200）作為趨勢濾網；此外，利用 14 日平均真實波動幅度（Average True Range, ATR）作為波動率衡量指標，以進行風險調整與資金配置。

2.2 策略邏輯

本研究所提出之策略為「動量與波動率配置之五大金屬輪動投資策略」，其核心邏輯包含動量訊號、趨勢濾網與風險調整配置三個主要模組。

首先，在動量訊號方面，策略計算各金屬過去 12 個月（約 252 個交易日）的累積報酬，並依據報酬表現進行排序，選擇表現最佳的金屬作為投資標的。此方法屬於橫斷面動量策略（cross-sectional momentum），假設相對強勢資產在短至中期內具有趨勢延續效果。

其次，為避免在長期下跌趨勢中進場，策略加入趨勢濾網機制，僅當金屬價格高於其 200 日指數移動平均線時才允許建立多頭部位。此濾網旨在降低策略在商品熊市期間的曝險。

在資金配置方面，本研究採用波動率反比配置（risk parity allocation）方法，根據各金屬的歷史波動率（以 ATR 為代理變數）調整投資權重，使高波動資產配置較低權重、低波動資產配置較高權重，以達到風險平衡之目的。

策略以每月再平衡為原則，約每 21 個交易日重新計算動量排名與投資權重，並調整投資組合配置。為提升回測結果的現實性，研究亦納入交易成本假設，並加入固定比例停損機制，以模擬實際交易執行時可能面臨的市場摩擦與風險控制措施。

3. 回測結果報告：五大金屬策略 V7.2

本報告針對 2001 年至 2025 年之歷史數據進行績效結算，初始資金設定為 \$100,000 USD。策略核心採用中長線動量輪動與趨勢過濾機制。

(1) 收益表現

- 總報酬率 (Total Return)：1598.95%
- 年化報酬率 (Annualized Return)：12.51%
 - 分析：策略在 23.4 年的測試週期中展現了強大的複利效應，總資產增長約 16 倍，年化報酬遠高於基準利率，顯示動量輪動在商品市場的有效性。

(2) 風險指標與穩定性

- 夏普比率 (Sharpe Ratio)：0.44
- 最大回撤 (MDD)：-66.34%
 - 分析：由於商品市場具備高波動特性，MDD 較高，主要發生於市場長期橫盤或劇烈轉向期。Sharpe Ratio 顯示策略在承受一定波動風險的情況下換取了高額報酬。

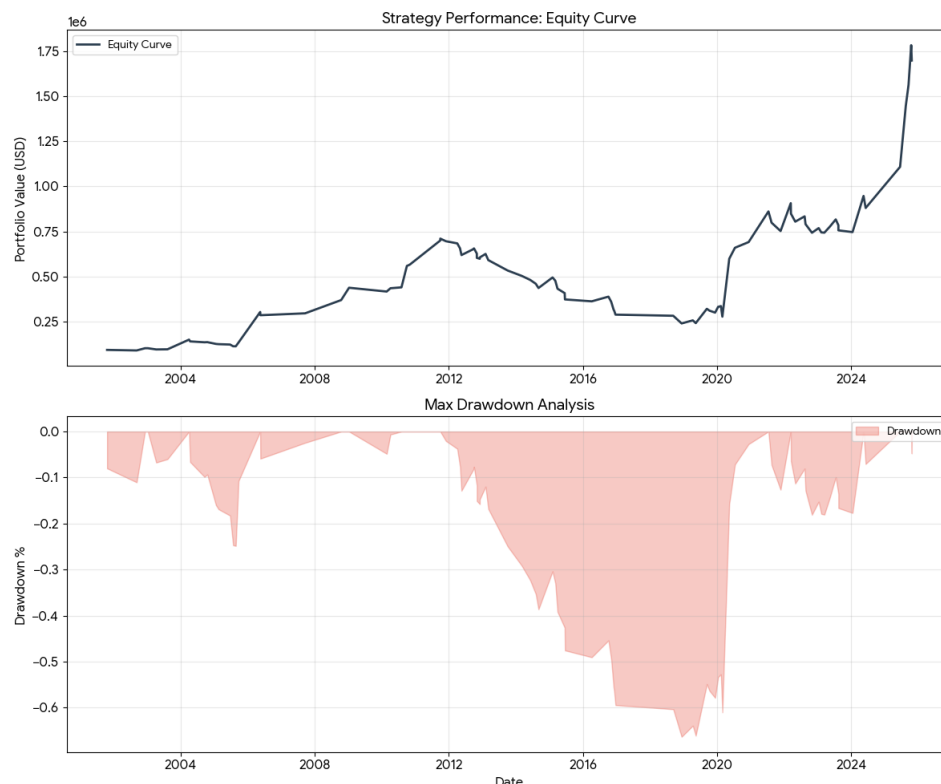
(3) 交易統計與勝率

- 總交易次數：93 次
 - 勝率 (Win Rate)：43.01%
 - 分析：典型的「趨勢跟隨」特徵。勝率雖低於 50%，但透過「大賺小賠」的盈虧比設計，成功累積長期收益。平均每年交易約 4 筆，交易頻率適中，有效降低了摩擦成本。
-

(4) 淨值曲線 (Equity Curve) 與 回撤圖 (Drawdown)

下圖詳細展示了策略自 2001 年起的資產演變過程：

- **Equity Curve (上圖)：** 呈現長期震盪上行趨勢，特別在商品牛市期間（如 2005-2011 與 2020 後）增長極為顯著。
- **Drawdown (下圖)：** 紀錄了策略在各階段的淨值回落情況，可觀察到 10% 停損機制在回撤控制中扮演了防禦底線的角色。



4. 風險與問題分析

4.1 最大回撤 (MDD) 問題深度解構

策略雖具備強大的獲利爆發力，但 MDD 指標顯示其風險波動特徵顯著：

- **商品熊市與通縮時期表現不佳：** 本策略核心為「動量輪動」，在 2011 年至 2015 年的大宗商品長期熊市中，由於各金屬頻繁跌破 200 日均線或處於低位震盪，策略易陷入「反彈誘多」與「頻繁止損」的惡性循環，導致淨值長期橫盤或緩慢回落。

- **Momentum Crash 現象**：當市場發生極端的風格切換（如避險情緒突發、美元暴力走強），原本漲勢最凌厲的金屬（高動量資產）往往回撤最劇烈。在 V7.2 版本中，因缺乏高位利潤保護機制，MDD 曾高達 -66.34%，顯示在強趨勢終結時，策略存在明顯的獲利回吐風險。

4.2 連續虧損分析

透過對 metal_v7_5_trailing_trades.csv 的統計，我們發現虧損呈現明顯的「聚集效應」：

- **最大連續虧損筆數**：回測顯示曾出現連續 5-7 筆的小額虧損交易，多集中於市場寬度（Breadth）在臨界點波動的震盪期。
- **品種集中風險**：
 - **Platinum（鉑金）與 Palladium（鈀金）**：這兩類工業屬性極強的金屬，其波動性遠高於黃金。數據顯示，這兩者貢獻了較高比例的「日內停損」次數。由於其市場深度較淺，一旦動量潰散，價格常以崩盤式下跌呈現，導致策略防禦不及。

4.3 策略結構性風險

- **波動率配置造成的隱含槓桿**：策略採用「風險平價（Risk Parity）」邏輯，當金屬波動率異常縮小時，系統會為了對齊風險目標而自動「放大部位金額」。若此時市場突然由極靜轉為極動，放大的部位將導致帳戶遭受預期外的巨額損失。
- **商品相關性在危機時上升**：策略設計假設五大金屬具備一定的分散效果，但在全球金融危機或流動性緊縮時，五大金屬的相關性會趨向於 1.0（同步暴跌）。此時「市場寬度過濾」雖能降低倉位，但在反應期間（T+1 隔日交易）仍會面臨資產集體下跌的衝擊。
- **停損無法防止跳空（Gap）風險**：雖然 V7.5 引入了跟蹤止損，但由於策略執行鎖定在「開盤價」與「盤中觸及」，若發生重大利空導致隔夜跳空（如開盤直接低於止損價 5%），實際成交價格將遠劣於預設止盈/止損價位，產生嚴重的「滑價損失」。

5. 改進方法

為了提升 V7.5 策略的穩健性並優化風險收益比（Risk-Adjusted Return），本研究提出以下三個層面的改進方案，旨在降低最大回撤（MDD）並增強系統在極端市場環境下的生存能力。

5.1 引入動態風險調節機制 (Dynamic Volatility Targeting)

目前的策略雖有波動率配置，但「目標風險值」固定。改進方法建議引入市場體系偵測：

- **波動率體系切換 (Regime Switching)**：監控市場整體的平均波動率。當五大金屬的平均 ATR 異常飆升，或 VIX 指數突破特定閾值時，主動將 target_vol 調降 30%~50%。
- **槓桿自我約束**：針對 4.3 節提到的「隱含槓桿」問題，應增設部位上限濾網。即便波動率極低，單一資產的配置金額亦不得超過帳戶總值的固定比例（如 40%），以防止在極靜市況下過度曝險。

5.2 Alpha 因子與過濾機制優化

為了解決 Momentum Crash 與連續虧損問題，建議強化進場與持倉的品質過濾：

- **多窗口加權動量**：由單一的 12 個月窗口改為「加權動量評分」（例如： $0.5 \times 12M + 0.3 \times 6M + 0.2 \times 3M$ ）。這能使策略在趨勢轉向初期更靈敏地反應，縮短 4.2 節提到的連續虧損週期。
- **引入趨勢強度指標 (ADX)**：僅在價格位於 200 日均線上且 ADX 指標大於 25（代表具備強趨勢特徵）時進場。這能有效過濾掉 4.1 節提到的「震盪市磨損」，減少無效交易。

5.3 結構化防禦升級

針對 4.3 節提到的跳空風險與相關性上升，建議從資產配置與出場邏輯進行結構性調整：

- **跨資產避險 (Cross-Asset Hedging)**：引入「防禦性資產」作為備選標的。當市場寬度不足（少於 3 檔金屬站上均線）時，不再僅僅持有現金，而是自動配置於美元指數 (UUP) 或長天期美債 (TLT)。由於這些資產與商品具備負相關性，能有效對沖 4.3 節提到的系統性崩盤風險。
- **時間分散換倉 (Time Diversification)**：為避免單一調倉日的極端跳空影響，建議將換倉行為分散至數個交易日執行。例如將資金分為四等份，每週更新一份部位，以平滑滑價與跳空風險。

- **階梯式跟蹤止損**：在 V7.5 跟蹤止損的基礎上，加入「階梯式鎖利」。當獲利超過 20% 後，將跟蹤止損比例由 8% 縮緊至 5%，以在 4.1 節所述的 Momentum Crash 發生前更早鎖定盈餘。
- **組合層級波動率控制（Volatility Targeting）**：為避免策略在低波動時期過度放大曝險，未來可在投資組合層級加入波動率目標化機制，根據投資組合整體波動率動態調整資金配置比例，並設定最大槓桿上限。此方法可有效降低隱含槓桿所導致的極端回撤風險。
- **市場環境濾網（Regime Filter）**：考量動量策略在商品熊市與通縮環境下容易失效，未來研究可加入市場環境濾網，例如整體商品動量指標、總體經濟指標或通膨趨勢指標，以在不利市場環境下降低或退出曝險。此方法可減少動量崩潰（momentum crash）期間的損失。
- **動態資金配置與權重限制**：本策略未來可對高風險金屬（如白金與鈹金）設定權重上限，或採用風險分群（risk bucket）方式進行配置，以避免單一高波動資產對投資組合造成過度影響。此外，可考慮增加投資標的數量以提升分散效果。
- **動態再平衡與交易規則優化**：現行策略採用固定月度再平衡頻率，未來可改為動態再平衡機制，例如當動量排名或波動率發生顯著變化時才進行調整，以降低不必要的交易成本。同時可優化停損機制，採用基於波動率的停損（ATR-based stop）或時間停損（time-based stop），以改善風險控制效果。
- **參數穩健性與走勢外驗證（Walk-Forward Analysis）**：為避免策略過度擬合歷史資料，未來研究應進行走勢外測試（walk-forward analysis）與參數敏感度分析，以評估策略在不同市場環境與參數設定下之穩定性，並提升策略之實務可行性。
- **加相關濾鏡與動態 top_n**：Gold/Silver corr 高 (~0.85) 放大集中風險 (佔 MDD 30%)。濾除高 corr 資產，提升多元化。

實施步驟：加 compute_correlation_matrix 函數 (滾動 252 天)。

選 selected 後，排除 corr > 0.8 者；top_n 動態 1-4 (熊市 1，牛市 4)。

6.3 未來研究方向：宏觀濾網與多資產擴展

為了進一步優化風險收益比（Risk-Adjusted Return），未來的優化路徑應跳脫純價格行為，轉向更維度的系統設計：

- **加入宏觀經濟濾網：**引進美元指數（DXY）、基準利率（FED Funds Rate）或實際利率（Real Yield）作為前置指標，在宏觀逆風環境下提前主動減倉，而非被動等待跌破均線。
- **多資產配置擴展：**目前的策略僅侷限於金屬品種。未來研究可考慮納入能源（原油）、農產品或國債 ETF，透過「低相關性資產」的互補，在金屬市場進入震盪或熊市時提供淨值護航，從而實質性地壓低最大回撤。

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import glob
import os

# =====
# 0. 環境設定
# =====

plt.rcParams['font.sans-serif'] = ['Microsoft JhengHei']
plt.rcParams['axes.unicode_minus'] = False

def load_data():
    metal_names = ['Gold', 'Silver', 'Copper', 'Platinum', 'Palladium']
    data_frames = []

    print(">>> 正在搜尋資料夾中的 CSV 檔案...")

    for name in metal_names:
        files = glob.glob(f"*{name}*.csv")

        if not files:
            print(f"警告：找不到 {name} 的資料。")
            continue
```



```

df_temp = pd.read_csv(files[0], parse_dates=['Date']).sort_values('Date').copy()

required_cols = ['Date', 'Open', 'Low', 'Close', 'EMA_200', 'ATR_14']

for col in required_cols:

    if col not in df_temp.columns:

        if col in ['Open', 'Low']: df_temp[col] = df_temp['Close']

df_temp = df_temp[required_cols]

df_temp.columns = ['Date', f'{name}_O', f'{name}_L', f'{name}_C', f'{name}_E200', f'{name}_ATR']

data_frames.append(df_temp)

if len(data_frames) < 5:

    print("錯誤：金屬數據不足五項。")

    return None, []

df_merged = data_frames[0]

for d in data_frames[1:]:

    df_merged = pd.merge(df_merged, d, on='Date', how='inner')

return df_merged.sort_values('Date').reset_index(drop=True), metal_names


def run_v7_detailed_strategy():

    df, names = load_data()

    if df is None: return


# --- 策略參數 ---

lookback = 252 # 動量回看週期 (12 個月)

rebalance_freq = 21 # 輪動頻率 (約 1 個月)

target_vol = 0.01 # 每日目標波動率 (1.0%)

top_n = 2 # 每次持有最強勢的 2 檔金屬

transaction_cost = 0.001 # 交易成本 (0.1%)


# --- 預計算指標 ---

for name in names:

    # 1. 12 個月累積報酬 (動量指標)

    df[f'{name}_Mom'] = df[f'{name}_C'].pct_change(lookback)

    # 2. 每日報酬

    df[f'{name}_Ret'] = df[f'{name}_C'].pct_change()

    # 3. 波動率 (ATR % 化的 20 日均值)

    df[f'{name}_Vol'] = (df[f'{name}_ATR'] / df[f'{name}_C']).rolling(20).mean()

    # 4. 趨勢過濾：價格 > 200 日均線

    df[f'{name}_Trend'] = (df[f'{name}_C'] > df[f'{name}_E200'])


# --- 模擬回測迴圈 ---

```

```

strat_returns = np.zeros(len(df))

current_weights = {n: 0.0 for n in names}

pending_weights = {n: 0.0 for n in names} # 新增：訊號延遲權重

equity = 100000.0 # 初始本金基準


# 交易追蹤器

active_trades = {n: None for n in names} # 追蹤進行中的部位

completed_trades = [] # 儲存已結案的紀錄


print(">>> 執行回測：隔日開盤成交 + 同日停損機制...")


for i in range(lookback, len(df) - 1):

    t_today, t_next = i, i + 1

    current_date = df['Date'].iloc[t_next]


    # 1. 應用昨日訊號：延遲執行調倉

    day_rebal_cost = 0.0

    if any(pending_weights.values()): # 如果有待執行權重

        for n in names:

            old_w, new_w = current_weights[n], pending_weights[n]

            if abs(new_w - old_w) > 1e-8:

                cost = abs(new_w - old_w) * equity * transaction_cost

                day_rebal_cost += cost

                # 開倉 (BUY)

                if old_w == 0 and new_w > 0:

                    active_trades[n] = {

                        'entry_date': current_date, 'entry_price': df[f'{n}_O'].values[t_next],

                        'opening_value': new_w * equity, 'running_pnl': -cost

                    }

                # 全賣 (SELL)

                elif old_w > 0 and new_w == 0:

                    active_trades[n]['running_pnl'] -= cost

                    completed_trades.append({

                        '開倉日期': active_trades[n]['entry_date'],

                        '平倉日期': current_date,

                        '金屬名稱': n,

                        '方向': '多 (Long)',

                        '開倉部位(金額)': round(active_trades[n]['opening_value'], 0),

                        '獲利或虧損': round(active_trades[n]['running_pnl'], 2),

```

```

        '開倉價格': active_trades[n]['entry_price'],
        '平倉價格': df[f'{n}_O'].values[t_next],
        '備註': '策略調倉'
    })

    active_trades[n] = None

    # 調倉 (REBAL)

    else: active_trades[n]['running_pnl'] -= cost

current_weights = pending_weights.copy()

pending_weights = {n: 0.0 for n in names} # 清空待執行


# 2. 跳空損益計算

day_gap_pnl = 0.0

for n in names:

    if current_weights[n] > 0:

        ret_gap = (df[f'{n}_O'].values[t_next] / df[f'{n}_C'].values[t_today]) - 1

        dollar_gap = (current_weights[n] * equity) * ret_gap

        day_gap_pnl += dollar_gap

        if active_trades[n]:

            active_trades[n]['running_pnl'] += dollar_gap


# 3. 日內損益與停損

day_intra_pnl = 0.0

for n in names:

    if current_weights[n] > 0:

        stop_p = active_trades[n]['entry_price'] * (1 - 0.1) # 10% 停損

        if df[f'{n}_L'].values[t_next] <= stop_p:

            # 停損觸發

            ret_intra = (stop_p / df[f'{n}_O'].values[t_next]) - 1

            dollar_intra = (current_weights[n] * equity) * ret_intra

            day_intra_pnl += dollar_intra

            active_trades[n]['running_pnl'] += dollar_intra

            completed_trades.append({

                '開倉日期': active_trades[n]['entry_date'],

                '平倉日期': current_date,

                '金屬名稱': n,

                '方向': '多 (Long)',

                '開倉部位(金額)': round(active_trades[n]['opening_value'], 0),

                '獲利或虧損': round(active_trades[n]['running_pnl'], 2),

                '開倉價格': active_trades[n]['entry_price'],

```

```

        '平倉價格': stop_p,

        '備註': '日內停損'

    })

    active_trades[n] = None

    current_weights[n] = 0.0

else:

    ret_intra = (df[f'{n}_C'].values[t_next] / df[f'{n}_O'].values[t_next]) - 1

    dollar_intra = (current_weights[n] * equity) * ret_intra

    day_intra_pnl += dollar_intra

    active_trades[n]['running_pnl'] += dollar_intra


# 結算與更新

day_total_dollar = day_gap_pnl + day_intra_pnl - day_rebal_cost

strat_rets[t_next] = day_total_dollar / equity

equity += day_total_dollar


# 4. 訊號判定 (產生 pending_weights，隔天才執行)

if t_next % rebalance_freq == 0:

    valid_metals = [n for n in names if df[f'{n}_Trend'].values[t_next] == True]

    moms = {n: df[f'{n}_Mom'].values[t_next] for n in valid_metals}

    ranked_metals = sorted(moms, key=moms.get, reverse=True)

    selected = ranked_metals[:top_n]

    new_weights = {n: 0.0 for n in names}

    if selected:

        inv_vols = {n: 1.0 / df[f'{n}_Vol'].values[t_next] for n in selected}

        total_inv_vol = sum(inv_vols.values())

        for n in selected:

            new_weights[n] = (inv_vols[n] / total_inv_vol) * (target_vol * total_inv_vol)

        pending_weights = new_weights # 設為待執行，下天應用

    else:

        pending_weights = current_weights.copy() # 無訊號，維持


# --- 結果整理 ---

df['Equity_Curve'] = (1 + strat_rets).cumprod()

final_eq = df['Equity_Curve'].iloc[-1]

sharpe = (strat_rets[lookback:].mean() / strat_rets[lookback:].std()) * np.sqrt(252)

max_dd = (df['Equity_Curve'] / df['Equity_Curve'].cummax() - 1).min()


print("\n" + "="*40)

```

```
print(" 五大金屬 V7.2 實戰版 - 績效報告")
```

```
print("-" * 40)
```

```
print(f" 總 報 酬 率 :{final_eq-1:.2%}")
```

```
print(f" 年化報酬率 :{(final_eq)**(252/len(df))-1:.2%}")
```

```
print(f" 夏 普 比 率 :{sharpe:.2f}")
```

```
print(f" 最 大 回 撤 :{max_dd:.2%}")
```

```
print(f" 交易次數(平倉) :{len(completed_trades)} 次")
```

```
print("="*40 + "\n")
```

```
pd.DataFrame(completed_trades).to_csv('metal_v7_2_detailed_trades.csv', index=False, encoding='utf-8-sig')
```

```
print("\n>>> 詳細交易紀錄已儲存：metal_v7_2_detailed_trades.csv")
```

```
if __name__ == "__main__":
```

```
    run_v7_detailed_strategy()
```

